

INDUSTRIAL AUTOMATION ENGINEERING PORTFOLIO

Project 9 — Version 2

Three-Floor Elevator Controller

PLC / Emulator Portfolio Project

Version 2 — Structured Controls Upgrade

1. Project Overview

Project 9 Version 2 is a PLC-controlled three-floor elevator system that manages floor requests, tracks the elevator's last confirmed floor, determines required travel direction, controls UP and DOWN movement, coordinates an automatic door sequence, supports Manual Mode, and detects abnormal operating conditions.

I developed the controller using an Allen-Bradley MicroLogix PLC in RSLogix Micro Starter Lite. The final architecture separates operator requests, stored states, direction decisions, motion permission, final movement commands, physical outputs, door sequencing, and diagnostics.

The final screenshot-confirmed ladder contains Rungs 0000–0037 followed by END at Rung 0038. The implementation includes Safety_OK I:1/13, Motion_Permissive B3:0/3, last-confirmed-floor tracking in N7:0, latched floor requests, automatic and manual direction commands, final UP/DOWN interlocks, automatic door control, direction-conflict monitoring, invalid-floor-sensor detection, travel supervision, master fault handling, and controlled recovery.

This is a PLC portfolio/emulator controller, not a certified passenger-elevator safety system.

2. Engineering Objective

- Store momentary floor requests until serviced.
- Track the last confirmed floor in N7:0.
- Determine automatic UP or DOWN direction from current floor and destination.
- Keep Automatic and Manual direction commands separate.
- Require System Ready, Door Closed, and no active door cycle before motion.
- Prevent simultaneous final UP and DOWN commands.
- Capture requested-floor arrival before clearing the request.
- Coordinate automatic door opening, dwell, closing, and completion.
- Detect direction conflict, invalid simultaneous floor sensors, and excessive travel time.
- Latch master faults and require controlled Reset followed by a separate Start.
- Clear stored floor requests during recovery.

3. Functional Requirements

- Start_PB I:1/0 shall latch Run_Request B3:0/0.
- Stop_PB I:1/1 or Fault_Active B3:0/2 shall unlatch Run_Request.
- System_Ready B3:0/1 shall require Run_Request, Safety_OK I:1/13, and XIO Fault_Active.
- Motion_Permissive B3:0/3 shall require System_Ready, Door_Closed I:1/9, and XIO Door_Cycle_Active B3:3/0.
- Floor sensors I:1/6–I:1/8 shall write 1, 2, or 3 to N7:0.
- Floor request pushbuttons I:1/3–I:1/5 shall latch B3:1/0–B3:1/2.
- Automatic UP and DOWN decisions shall be inhibited when Manual_Mode I:1/10 is true.
- Manual_Up_PB I:1/11 and Manual_Down_PB I:1/12 shall generate B3:2/2 and B3:2/3 in Manual Mode.
- Final_Up_Command B3:2/4 and Final_Down_Command B3:2/5 shall use Motion_Permissive and opposite-direction interlocks.
- Motor_Up O:2/0 and Motor_Down O:2/1 shall be driven only by the corresponding final commands.
- Requested-floor arrival shall latch Door_Cycle_Active B3:3/0 before the serviced request is cleared.
- Door_Open O:2/2 shall energize while Door_Cycle_Active is true and T4:0/DN is false.
- T4:0 shall use TB 1.0 and PRE 20.
- Door_Close O:2/3 shall energize after T4:0/DN while Door_Closed is false.
- Door_Cycle_Active shall unlatch only when T4:0/DN and Door_Closed are both true.
- Direction_Conflict B3:4/0 shall monitor contradictory Auto or Manual direction-command pairs.
- Invalid_Floor_Sensors B3:4/1 shall detect any two floor sensors simultaneously true.

- T4:1 shall supervise final movement commands using TB 1.0 and PRE 60.
- T4:1/DN shall latch Travel_Timeout B3:4/2.
- Direction_Conflict, Invalid_Floor_Sensors, or Travel_Timeout shall latch Fault_Active B3:0/2.
- Reset_PB I:1/2 shall clear Fault_Active, all stored floor requests, and Travel_Timeout.

4. System Components

Component	Address	Function
Start Push Button	I:1/0	Latches Run Request
Stop Push Button	I:1/1	Removes Run Request
Reset Push Button	I:1/2	Fault/request recovery
Floor 1 Request PB	I:1/3	Requests Floor 1
Floor 2 Request PB	I:1/4	Requests Floor 2
Floor 3 Request PB	I:1/5	Requests Floor 3
Floor 1 Sensor	I:1/6	Confirms Floor 1
Floor 2 Sensor	I:1/7	Confirms Floor 2
Floor 3 Sensor	I:1/8	Confirms Floor 3
Door Closed	I:1/9	Door-closed feedback
Manual Mode	I:1/10	Selects Manual Mode
Manual Up PB	I:1/11	Manual upward request
Manual Down PB	I:1/12	Manual downward request
Safety OK	I:1/13	Safety/permissive input
Motor Up	O:2/0	Upward motor command
Motor Down	O:2/1	Downward motor command
Door Open	O:2/2	Door opening command
Door Close	O:2/3	Door closing command
Fault Indicator	O:2/4	Master fault indication
Door Dwell Timer	T4:0	TB 1.0, PRE 20
Travel Supervision Timer	T4:1	TB 1.0, PRE 60
Current Floor	N7:0	Last confirmed floor

5. PLC Data and Address Assignment

Address	Symbolic Name	Function
B3:0/0	Run_Request	Latched operating request
B3:0/1	System_Ready	Ready status
B3:0/2	Fault_Active	Latched master fault
B3:0/3	Motion_Permissive	Common movement permission
B3:1/0	Floor_1_Request	Stored Floor 1 request
B3:1/1	Floor_2_Request	Stored Floor 2 request
B3:1/2	Floor_3_Request	Stored Floor 3 request
B3:2/0	Auto_Up_Command	Automatic upward decision
B3:2/1	Auto_Down_Command	Automatic downward decision
B3:2/2	Manual_Up_Command	Manual upward command
B3:2/3	Manual_Down_Command	Manual downward command
B3:2/4	Final_Up_Command	Final internal UP command
B3:2/5	Final_Down_Command	Final internal DOWN command
B3:3/0	Door_Cycle_Active	Automatic door-cycle state
B3:4/0	Direction_Conflict	Direction diagnostic

B3:4/1	Invalid_Floor_Sensors	Position diagnostic
B3:4/2	Travel_Timeout	Latched travel timeout
T4:0	Door_Dwell_Timer	TON, TB 1.0, PRE 20
T4:1	Travel_Supervision_Timer	TON, TB 1.0, PRE 60
N7:0	Current_Floor	Last confirmed floor

6. Control Strategy

I organized the controller as Operator Request → Stored State → Position Evaluation → Direction Decision → Motion Permission → Final Command → Physical Output. A separate diagnostic path handles abnormal conditions through individual diagnostics, a latched master fault, controlled shutdown, and deliberate recovery.

Start latches Run Request, while Stop or a master fault removes it. System Ready additionally requires Safety_OK. Motion Permissive is separate again and requires a closed door with no active door cycle.

Floor sensors write the last confirmed position into N7:0. Stored requests remain active after the pushbutton is released. Automatic direction rungs compare the current floor to the requested destination, while Manual Mode uses independent UP and DOWN commands.

Both Automatic and Manual movement requests converge at the final-command layer, where Motion Permissive and opposite-direction interlocks are applied. Arrival is captured before request clearing, then Door_Cycle_Active controls the door open/dwell/close sequence.

T4:1 supervises any final movement command. A timeout is latched so the diagnostic remains visible after the master fault stops movement and resets the non-retentive timer.

7. Engineering Design Decisions

Design Decision	Engineering Purpose
Latched Run Request	Maintains a momentary Start command
Safety_OK in System Ready	Adds independent operating permission
System Ready separate from Run Request	Separates operator intent from PLC authorization
Motion Permissive	Centralizes door-related movement permission
Door Cycle Active interlock	Prevents movement during door operation
N7:0 Current Floor	Retains last confirmed floor between sensors
Latched floor requests	Stores destination after PB release
Automatic direction bits	Separate decision from output
Manual command bits	Separate manual requests from Auto logic
Final UP/DOWN command layer	Applies shared permissive and mutual exclusion
Arrival before request clear	Uses scan order to preserve destination arrival
Door Cycle Active state	Retains automatic door sequence
Feedback-based cycle completion	Requires Door_Closed before motion returns
Direction Conflict diagnostic	Detects contradictory upstream requests
Invalid floor sensor diagnostic	Detects impossible simultaneous position feedback
Travel timeout latch	Preserves cause after motion stops
Reset clears stored requests	Prevents stale post-fault movement
Separate Start after Reset	Prevents automatic restart

8. Ladder Logic Description

8.1 Rung 0000 — Start / Latch Run Request

Ladder Logic

```
|-----[ XIC I:1/0 ]----- ( OTL B3:0/0 )-----|  
      Start_PB                      Run_Request
```

Rung Description:

Rung 0000 latches Run_Request B3:0/0 when Start_PB I:1/0 is pressed. Because the rung uses an OTL instruction, the run request stays ON after the momentary Start pushbutton is released.

Engineering Purpose:

I separated the momentary Start input from the maintained operating request. This gives the rest of the controller a stable internal run state instead of depending on the physical pushbutton remaining pressed.

8.2 Rung 0001 — Stop / Unlatch Run Request

Ladder Logic

```
|-----[ XIC I:1/1 ]----- ( OTU B3:0/0 )-----|  
      Stop_PB                      Run_Request
```

Rung Description:

Rung 0001 clears Run_Request B3:0/0 when Stop_PB I:1/1 is pressed. The OTU instruction removes the stored run request immediately.

Engineering Purpose:

I kept the Stop command as a direct unlatch of the stored run request. Once Stop is pressed, the controller no longer has permission to remain in its running state.

8.3 Rung 0002 — Fault Removes Run Request

Ladder Logic

```
|-----[ XIC B3:0/2 ]----- ( OTU B3:0/0 )-----|  
      Fault_Active                      Run_Request
```

Rung Description:

Rung 0002 also unlatches Run_Request B3:0/0, but this time from Fault_Active B3:0/2. As soon as the master fault becomes true, the stored run request is removed.

Engineering Purpose:

I wanted a fault to do more than temporarily block a motor output. Removing Run_Request forces the controller back through its normal ready/start path after recovery, so a separate Start command is required before movement can resume.

8.4 Rung 0003 — System Ready

Ladder Logic

```
|-----[ XIC B3:0/0 ]-----[ XIC I:1/13 ]-----[ XIO B3:0/2 ]----- ( OTE B3:0/1 )-----|  
      Run_Request          Safety_OK          Fault_Active          System_Ready
```

Rung Description:

Rung 0003 generates System_Ready B3:0/1. Run_Request B3:0/0 and Safety_OK I:1/13 must both be true, while Fault_Active B3:0/2 must be OFF. Because this is an OTE rung, System_Ready drops out immediately if any of those conditions is lost.

Engineering Purpose:

Run_Request only shows that the operator wants the controller active; System_Ready confirms that the safety permissive is present and no master fault exists. Keeping those conditions separate makes the operating state clearer.

8.5 Rung 0004 — Motion Permissive

Ladder Logic

```
|----[ XIC B3:0/1 ]----[ XIC I:1/9 ]----[ XIO B3:3/0 ]-----[ OTE B3:0/3 ]----|  
    System_Ready      Door_Closed      Door_Cycle_Active      Motion_Permissive
```

Rung Description:

Rung 0004 generates Motion_Permissive B3:0/3. System_Ready B3:0/1 and Door_Closed I:1/9 must be true, and Door_Cycle_Active B3:3/0 must be OFF. If the door opens or a door cycle starts, Motion_Permissive turns OFF.

Engineering Purpose:

I made door status part of a common movement permissive instead of repeating the same contacts in every UP and DOWN path. The added XIO Door_Cycle_Active contact prevents motion during the complete door sequence, even if Door_Closed feedback is still present during simulation.

8.6 Rung 0005 — Floor 1 Position Update

Ladder Logic

```
|----[ XIC I:1/6 ]-----[ MOV 1 N7:0 ]----|  
    Floor_1_Sensor              Current_Floor
```

Rung Description:

Rung 0005 writes the value 1 to N7:0 Current_Floor whenever Floor_1_Sensor I:1/6 is true. The MOV instruction updates the stored floor number only while Floor 1 is confirmed.

Engineering Purpose:

I use N7:0 as the last confirmed floor rather than continuous position. Once the elevator leaves Floor 1 and the sensor turns OFF, the value 1 remains stored until another floor sensor updates the register.

8.7 Rung 0006 — Floor 2 Position Update

Ladder Logic

```
|----[ XIC I:1/7 ]-----[ MOV 2 N7:0 ]----|  
    Floor_2_Sensor              Current_Floor
```

Rung Description:

Rung 0006 writes the value 2 to N7:0 when Floor_2_Sensor I:1/7 is true.

Engineering Purpose:

This keeps the same last-confirmed-floor method for Floor 2. The direction logic can then use a single integer register instead of checking every floor sensor combination directly.

8.8 Rung 0007 — Floor 3 Position Update

Ladder Logic

```
|----[ XIC I:1/8 ]-----[ MOV 3 N7:0 ]----|  
    Floor_3_Sensor           Current_Floor
```

Rung Description:

Rung 0007 writes the value 3 to N7:0 when Floor_3_Sensor I:1/8 is true. When the elevator is between floors and all three floor sensors are OFF, N7:0 keeps the most recently written value.

Engineering Purpose:

Retaining the last confirmed floor gives the automatic direction logic a stable reference while the car is traveling between discrete floor sensors.

8.9 Rung 0008 — Latch Floor 1 Request

Ladder Logic

```
|----[ XIC I:1/3 ]----- ( OTL B3:1/0 )----|  
    Floor_1_Request_PB           Floor_1_Request
```

Rung Description:

Rung 0008 latches Floor_1_Request B3:1/0 when Floor_1_Request_PB I:1/3 is pressed. The OTL keeps the request stored after the pushbutton is released.

Engineering Purpose:

A passenger request has to remain active until the destination is serviced. Storing the request internally allows the controller to remember Floor 1 without requiring the pushbutton to stay ON.

8.10 Rung 0009 — Latch Floor 2 Request

Ladder Logic

```
|----[ XIC I:1/4 ]----- ( OTL B3:1/1 )----|  
    Floor_2_Request_PB           Floor_2_Request
```

Rung Description:

Rung 0009 latches Floor_2_Request B3:1/1 from Floor_2_Request_PB I:1/4.

Engineering Purpose:

This gives Floor 2 the same retained-request behavior as Floor 1. The request stays active until the arrival logic or Reset clears it.

8.11 Rung 0010 — Latch Floor 3 Request

Ladder Logic

```
|----[ XIC I:1/5 ]----- ( OTL B3:1/2 )----|  
    Floor_3_Request_PB           Floor_3_Request
```

Rung Description:

Rung 0010 latches Floor_3_Request B3:1/2 from Floor_3_Request_PB I:1/5.

Engineering Purpose:

Floor 3 requests are stored in the same way as the other floors so all destinations use one consistent request-handling method.

8.12 Rung 0011 — Automatic UP Direction Decision

Ladder Logic

```
|-----[ XIO I:1/10 ]-----+-----[ EQU N7:0 1 ]-----[ XIC B3:1/1 ]-----+----- ( OTE B3:2/0 )-----|
      Manual_Mode              |      Current_Floor=1      Floor_2_Request |      Auto_Up_Command
                              +-----[ EQU N7:0 1 ]-----[ XIC B3:1/2 ]-----+
                              |      Current_Floor=1      Floor_3_Request |
                              +-----[ EQU N7:0 2 ]-----[ XIC B3:1/2 ]-----+
                              |      Current_Floor=2      Floor_3_Request |
```

Rung Description:

Rung 0011 determines when automatic UP travel is required. Manual_Mode I:1/10 must be OFF. The three parallel branches cover Current_Floor N7:0 = 1 with a Floor 2 request, Current_Floor = 1 with a Floor 3 request, and Current_Floor = 2 with a Floor 3 request. Any valid branch energizes Auto_Up_Command B3:2/0.

Engineering Purpose:

I separated the decision to travel UP from the final motor command. This rung only answers the question of direction; the later final-command rung still has to apply the common motion permissive and opposite-direction interlock.

8.13 Rung 0012 — Automatic DOWN Direction Decision

Ladder Logic

```
|-----[ XIO I:1/10 ]-----+-----[ EQU N7:0 3 ]-----[ XIC B3:1/1 ]-----+----- ( OTE B3:2/1 )-----|
      Manual_Mode              |      Current_Floor=3      Floor_2_Request |      Auto_Down_Command
                              +-----[ EQU N7:0 3 ]-----[ XIC B3:1/0 ]-----+
                              |      Current_Floor=3      Floor_1_Request |
                              +-----[ EQU N7:0 2 ]-----[ XIC B3:1/0 ]-----+
                              |      Current_Floor=2      Floor_1_Request |
```

Rung Description:

Rung 0012 determines automatic DOWN travel while Manual_Mode I:1/10 is OFF. The branches cover Current_Floor N7:0 = 3 with a Floor 2 request, Current_Floor = 3 with a Floor 1 request, and Current_Floor = 2 with a Floor 1 request. Any true branch energizes Auto_Down_Command B3:2/1.

Engineering Purpose:

This mirrors the UP decision logic for destinations below the current floor. Keeping UP and DOWN decisions in separate rungs makes the direction selection explicit before the physical outputs are considered.

8.14 Rung 0013 — Final UP Command

Ladder Logic

```
|-----+-----[ XIC B3:2/0 ]-----+-----[ XIC B3:0/3 ]-----+-----[ XIO B3:2/5 ]-----+----- ( OTE B3:2/4 )-----|
      |      Auto_Up_Command |      Motion_Permissive      Final_Down_Command      Final_Up_Command
      +-----[ XIC B3:2/2 ]-----+
      |      Manual_Up_Command |
```

Rung Description:

Rung 0013 generates Final_Up_Command B3:2/4. Either Auto_Up_Command B3:2/0 or Manual_Up_Command B3:2/2 can request upward travel, but Motion_Permissive B3:0/3 must be true and Final_Down_Command B3:2/5 must be OFF.

Engineering Purpose:

I combined Automatic and Manual UP requests at one final command point. Both operating modes therefore use the same door/ready permissive and the same protection against simultaneous opposite-direction commands.

8.15 Rung 0014 — Final DOWN Command

Ladder Logic

```
|-----+-----[ XIC B3:2/1 ]-----+-----[ XIC B3:0/3 ]-----+-----[ XIO B3:2/4 ]-----+----- ( OTE B3:2/5 )-----|
|      Auto_Down_Command |           Motion_Permissive   Final_Up_Command   Final_Down_Command
+-----[ XIC B3:2/3 ]-----+
      Manual_Down_Command
```

Rung Description:

Rung 0014 generates Final_Down_Command B3:2/5 from either Auto_Down_Command B3:2/1 or Manual_Down_Command B3:2/3. Motion_Permissive B3:0/3 must be true, and Final_Up_Command B3:2/4 must be OFF.

Engineering Purpose:

This is the matching final command layer for downward movement. The actual motor output is not allowed to depend directly on either Automatic or Manual requests until the common movement conditions are satisfied.

8.16 Rung 0015 — Motor UP Output

Ladder Logic

```
|-----[ XIC B3:2/4 ]-----+----- ( OTE O:2/0 )-----|
      Final_Up_Command           Motor_Up
```

Rung Description:

Rung 0015 energizes Motor_Up O:2/0 whenever Final_Up_Command B3:2/4 is true. When the final command drops out, the output turns OFF.

Engineering Purpose:

I kept the physical UP output simple because all direction decisions, mode selection, door conditions, and interlocks have already been resolved before this rung.

8.17 Rung 0016 — Motor DOWN Output

Ladder Logic

```
|-----[ XIC B3:2/5 ]-----+----- ( OTE O:2/1 )-----|
      Final_Down_Command           Motor_Down
```

Rung Description:

Rung 0016 energizes Motor_Down O:2/1 from Final_Down_Command B3:2/5.

Engineering Purpose:

The DOWN motor output follows the same final-command structure as the UP output, so there is one clear internal signal responsible for each physical direction output.

8.18 Rung 0017 — Capture Requested-Floor Arrival

Ladder Logic

```
|-----+-----[ XIC I:1/6 ]-----[ XIC B3:1/0 ]-----+----- ( OTL B3:3/0 )-----|
|      Floor_1_Sensor      Floor_1_Request |      Door_Cycle_Active
+-----[ XIC I:1/7 ]-----[ XIC B3:1/1 ]-----+
|      Floor_2_Sensor      Floor_2_Request |
+-----[ XIC I:1/8 ]-----[ XIC B3:1/2 ]-----+
|      Floor_3_Sensor      Floor_3_Request
```

Rung Description:

Rung 0017 captures arrival at a requested floor. Any matching pair of floor sensor and stored request can latch Door_Cycle_Active B3:3/0: Floor 1 sensor with Floor 1 request, Floor 2 sensor with Floor 2 request, or Floor 3 sensor with Floor 3 request.

Engineering Purpose:

I placed this rung before the request-clearing rungs so the PLC captures the arrival while the stored request is still true. Door_Cycle_Active then preserves the arrival event after the request is cleared later in the same scan.

8.19 Rung 0018 — Clear Floor 1 Request at Arrival

Ladder Logic

```
|-----[ XIC I:1/6 ]-----[ XIC B3:1/0 ]----- ( OTU B3:1/0 )-----|
|      Floor_1_Sensor      Floor_1_Request      Floor_1_Request
```

Rung Description:

Rung 0018 unlatches Floor_1_Request B3:1/0 when Floor_1_Sensor I:1/6 and the stored Floor 1 request are both true.

Engineering Purpose:

Once Floor 1 has been reached, that destination no longer needs to remain in request memory. Clearing it here prevents the serviced floor from continuing to influence the direction logic.

8.20 Rung 0019 — Clear Floor 2 Request at Arrival

Ladder Logic

```
|-----[ XIC I:1/7 ]-----[ XIC B3:1/1 ]----- ( OTU B3:1/1 )-----|
|      Floor_2_Sensor      Floor_2_Request      Floor_2_Request
```

Rung Description:

Rung 0019 clears Floor_2_Request B3:1/1 when Floor_2_Sensor I:1/7 confirms arrival while the request is active.

Engineering Purpose:

This removes the completed Floor 2 destination after the door-cycle capture rung has already stored the arrival event.

8.21 Rung 0020 — Clear Floor 3 Request at Arrival

Ladder Logic

```
|-----[ XIC I:1/8 ]-----[ XIC B3:1/2 ]----- ( OTU B3:1/2 )-----|
|      Floor_3_Sensor      Floor_3_Request      Floor_3_Request
```

Rung Description:

Rung 0020 clears Floor_3_Request B3:1/2 when Floor_3_Sensor I:1/8 and the stored Floor 3 request are both true.

Engineering Purpose:

This completes the same request-service pattern for Floor 3 and keeps the destination memory synchronized with actual arrivals.

8.22 Rung 0021 — Door Open Output**Ladder Logic**

```
|----[ XIC B3:3/0 ]-----[ XIO T4:0/DN ]----- ( OTE O:2/2 )----|
      Door_Cycle_Active   Door_Timer_Done           Door_Open
```

Rung Description:

Rung 0021 controls Door_Open O:2/2. Door_Cycle_Active B3:3/0 must be true and T4:0/DN must still be OFF. The door-open output turns OFF when the dwell timer reaches Done.

Engineering Purpose:

I use the timer Done bit to define the end of the open/dwell portion of the door cycle. The door remains commanded open for the full programmed dwell period instead of relying on a momentary arrival condition.

8.23 Rung 0022 — Door Dwell Timer**Ladder Logic**

```
|----[ XIC B3:3/0 ]-----[ TON T4:0 ]----|
      Door_Cycle_Active           Door_Dwell_Timer
                                   Time Base: 1.0
                                   Preset: 20
```

Rung Description:

Rung 0022 runs Door_Dwell_Timer T4:0 whenever Door_Cycle_Active B3:3/0 is true. The final frozen ladder uses a 1.0-second time base and a preset of 20.

Engineering Purpose:

The longer 20-second value was selected for emulator commissioning so the door sequence could be observed and the feedback could be simulated manually. The timer provides a repeatable dwell interval before the controller begins closing the door.

8.24 Rung 0023 — Door Close Output**Ladder Logic**

```
|----[ XIC B3:3/0 ]-----[ XIC T4:0/DN ]----[ XIO I:1/9 ]----- ( OTE O:2/3 )----|
      Door_Cycle_Active   Timer_Done           Door_Closed           Door_Close
```

Rung Description:

Rung 0023 energizes Door_Close O:2/3 after T4:0/DN becomes true. Door_Cycle_Active must still be true, and Door_Closed I:1/9 must remain false. Once the closed feedback turns ON, the XIO contact opens and Door_Close turns OFF.

Engineering Purpose:

I made door closing depend on actual Door_Closed feedback rather than another fixed delay. The controller continues commanding close only until the closed condition is confirmed.

8.25 Rung 0024 — Complete Door Cycle

Ladder Logic

```
|-----[ XIC B3:3/0 ]-----[ XIC T4:0/DN ]-----[ XIC I:1/9 ]----- ( OTU B3:3/0 )-----|  
Door_Cycle_Active      Timer_Done      Door_Closed      Door_Cycle_Active
```

Rung Description:

Rung 0024 completes the door cycle. Door_Cycle_Active B3:3/0, T4:0/DN, and Door_Closed I:1/9 must all be true before the OTU clears Door_Cycle_Active.

Engineering Purpose:

Motion should not be restored simply because the dwell timer expired. I require both timer completion and closed-door confirmation before ending the door cycle; clearing Door_Cycle_Active then allows Motion_Permissive to return.

8.26 Rung 0025 — Manual UP Command

Ladder Logic

```
|-----[ XIC I:1/10 ]-----[ XIC I:1/11 ]----- ( OTE B3:2/2 )-----|  
Manual_Mode      Manual_Up_PB      Manual_Up_Command
```

Rung Description:

Rung 0025 generates Manual_Up_Command B3:2/2 while Manual_Mode I:1/10 and Manual_Up_PB I:1/11 are both true. Since this is an OTE rung, the command drops out when the pushbutton is released.

Engineering Purpose:

I kept manual UP operation hold-to-run. Manual Mode changes the source of the movement request, but the command still has to pass through the same final UP rung and Motion_Permissive as automatic travel.

8.27 Rung 0026 — Manual DOWN Command

Ladder Logic

```
|-----[ XIC I:1/10 ]-----[ XIC I:1/12 ]----- ( OTE B3:2/3 )-----|  
Manual_Mode      Manual_Down_PB      Manual_Down_Command
```

Rung Description:

Rung 0026 generates Manual_Down_Command B3:2/3 while Manual_Mode I:1/10 and Manual_Down_PB I:1/12 are true.

Engineering Purpose:

Manual DOWN follows the same hold-to-run approach as Manual UP and still uses the common final-command interlocks before Motor_Down can energize.

8.28 Rung 0027 — Direction Conflict Diagnostic

Ladder Logic

```
|-----+-----[ XIC B3:2/0 ]-----[ XIC B3:2/1 ]-----+----- ( OTE B3:4/0 )-----|  
|      Auto_Up_Command      Auto_Down_Command |      Direction_Conflict  
+-----[ XIC B3:2/2 ]-----[ XIC B3:2/3 ]-----+  
Manual_Up_Command      Manual_Down_Command
```

Rung Description:

Rung 0027 generates Direction_Conflict B3:4/0 if Auto_Up_Command B3:2/0 and Auto_Down_Command B3:2/1 are both true, or if Manual_Up_Command B3:2/2 and Manual_Down_Command B3:2/3 are both true. Because the rung uses an OTE, the diagnostic reflects the current conflicting command condition.

Engineering Purpose:

The final UP and DOWN command rungs already prevent both motor directions from being energized together. I added this separate diagnostic so contradictory requests are detected and reported instead of being silently blocked by the output interlocks.

8.29 Rung 0028 — Invalid Floor Sensor Diagnostic

Ladder Logic

```
|-----+-----[ XIC I:1/6 ]-----+-----[ XIC I:1/7 ]-----+----- ( OTE B3:4/1 )-----|
|      Floor_1_Sensor      Floor_2_Sensor      |      Invalid_Floor_Sensors
+-----+-----[ XIC I:1/6 ]-----+-----[ XIC I:1/8 ]-----+
|      Floor_1_Sensor      Floor_3_Sensor      |
+-----+-----[ XIC I:1/7 ]-----+-----[ XIC I:1/8 ]-----+
|      Floor_2_Sensor      Floor_3_Sensor
```

Rung Description:

Rung 0028 checks for invalid simultaneous floor feedback. Any pair of Floor_1_Sensor I:1/6, Floor_2_Sensor I:1/7, and Floor_3_Sensor I:1/8 being true at the same time energizes Invalid_Floor_Sensors B3:4/1.

Engineering Purpose:

The elevator should only be confirmed at one floor at a time. This diagnostic catches a sensor, wiring, or simulation condition that would make the position information physically inconsistent.

8.30 Rung 0029 — Travel Supervision Timer

Ladder Logic

```
|-----+-----[ XIC B3:2/4 ]-----+-----+-----[ TON T4:1 ]-----|
|      Final_Up_Command      |      Travel_Supervision_Timer
+-----+-----[ XIC B3:2/5 ]-----+-----+-----
|      Final_Down_Command      |      Time Base: 1.0
|                               |      Preset: 60
```

Rung Description:

Rung 0029 runs Travel_Supervision_Timer T4:1 whenever either Final_Up_Command B3:2/4 or Final_Down_Command B3:2/5 is true. The final frozen ladder uses a 1.0-second time base and a preset of 60.

Engineering Purpose:

I supervise the final movement command rather than the original floor request so the timer measures actual commanded travel. The 60-second preset was chosen for emulator commissioning to allow enough time for manual floor-sensor simulation.

8.31 Rung 0030 — Latch Travel Timeout

Ladder Logic

```
|-----[ XIC T4:1/DN ]-----+----- ( OTL B3:4/2 )-----|
|      Travel_Timer_Done      |      Travel_Timeout
```

Rung Description:

Rung 0030 latches Travel_Timeout B3:4/2 when T4:1/DN becomes true. Because an OTL is used, the timeout remains stored even after the master fault removes movement and the non-retentive travel timer resets.

Engineering Purpose:

I needed the cause of an excessive-travel shutdown to remain visible after the motor command is removed. Latching Travel_Timeout preserves that diagnostic until the operator performs a Reset.

8.32 Rung 0031 — Master Fault Latch

Ladder Logic

```
|-----[ XIC B3:4/0 ]-----+----- ( OTL B3:0/2 )-----|
|      Direction_Conflict|      Fault_Active
+----[ XIC B3:4/1 ]-----+
|      Invalid_Floor_Sensors
+----[ XIC B3:4/2 ]-----+
|      Travel_Timeout
```

Rung Description:

Rung 0031 latches Fault_Active B3:0/2 if Direction_Conflict B3:4/0, Invalid_Floor_Sensors B3:4/1, or Travel_Timeout B3:4/2 is true. Any one of the three parallel branches can set the master fault.

Engineering Purpose:

I kept the individual diagnostics separate so the cause can be identified, but I use one master fault to produce the common shutdown response. Fault_Active then removes Run_Request through Rung 0002 and stops movement through the normal control hierarchy.

8.33 Rung 0032 — Reset Master Fault

Ladder Logic

```
|----[ XIC I:1/2 ]-----+----- ( OTU B3:0/2 )-----|
|      Reset_PB|      Fault_Active
```

Rung Description:

Rung 0032 clears Fault_Active B3:0/2 when Reset_PB I:1/2 is pressed. The OTU removes the latched master fault.

Engineering Purpose:

Reset acknowledges the stored master fault, but it does not restart the elevator. Because Run_Request was removed when the fault occurred, the operator must still press Start after the abnormal condition has been corrected.

8.34 Rung 0033 — Fault Indicator

Ladder Logic

```
|----[ XIC B3:0/2 ]-----+----- ( OTE O:2/4 )-----|
|      Fault_Active|      Fault_Indicator
```

Rung Description:

Rung 0033 energizes Fault_Indicator O:2/4 whenever Fault_Active B3:0/2 is true.

Engineering Purpose:

The physical indicator follows the master fault so the operator has a direct visual indication that the controller is in its faulted state.

8.35 Rung 0034 — Reset Floor 1 Request

Ladder Logic

```
|----[ XIC I:1/2 ]----- ( OTU B3:1/0 )----|  
Reset_PB                      Floor_1_Request
```

Rung Description:

Rung 0034 clears Floor_1_Request B3:1/0 when Reset_PB I:1/2 is pressed.

Engineering Purpose:

I clear stored destination requests during recovery so an old Floor 1 request cannot cause unexpected movement after the controller is restarted.

8.36 Rung 0035 — Reset Floor 2 Request

Ladder Logic

```
|----[ XIC I:1/2 ]----- ( OTU B3:1/1 )----|  
Reset_PB                      Floor_2_Request
```

Rung Description:

Rung 0035 clears Floor_2_Request B3:1/1 from the same Reset pushbutton.

Engineering Purpose:

This removes any stale Floor 2 destination as part of the fault/request recovery process.

8.37 Rung 0036 — Reset Floor 3 Request

Ladder Logic

```
|----[ XIC I:1/2 ]----- ( OTU B3:1/2 )----|  
Reset_PB                      Floor_3_Request
```

Rung Description:

Rung 0036 clears Floor_3_Request B3:1/2 when Reset_PB I:1/2 is pressed.

Engineering Purpose:

All three stored destinations are returned to a known state during Reset so the next operating cycle begins without leftover passenger requests.

8.38 Rung 0037 — Reset Travel Timeout

Ladder Logic

```
|----[ XIC I:1/2 ]----- ( OTU B3:4/2 )----|  
Reset_PB                      Travel_Timeout
```

Rung Description:

Rung 0037 clears the latched Travel_Timeout B3:4/2 when Reset_PB I:1/2 is pressed.

Engineering Purpose:

Travel_Timeout is intentionally retained after the shutdown so it can be diagnosed. This rung removes it only when the operator performs the recovery action.

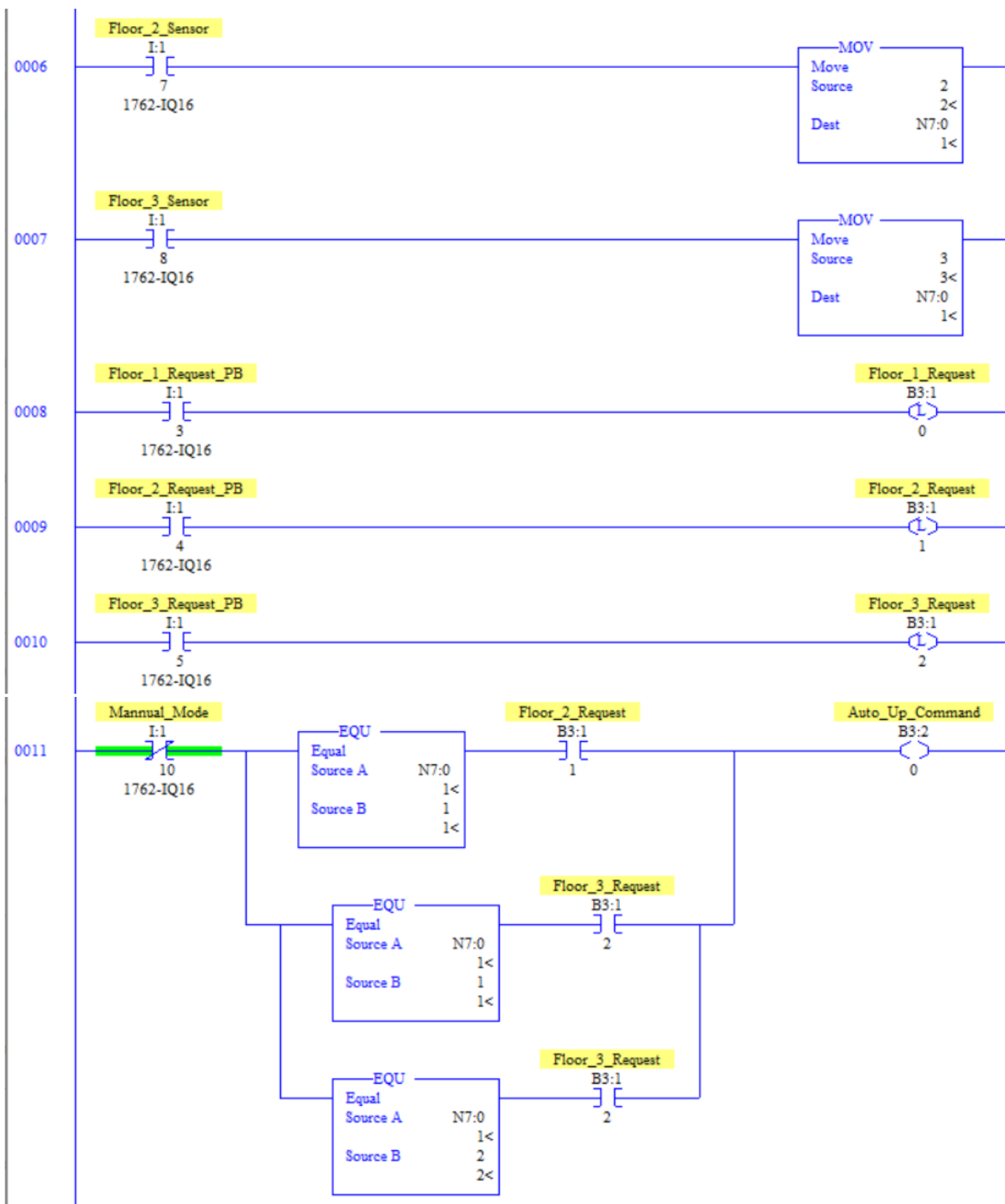
Ladder Logic

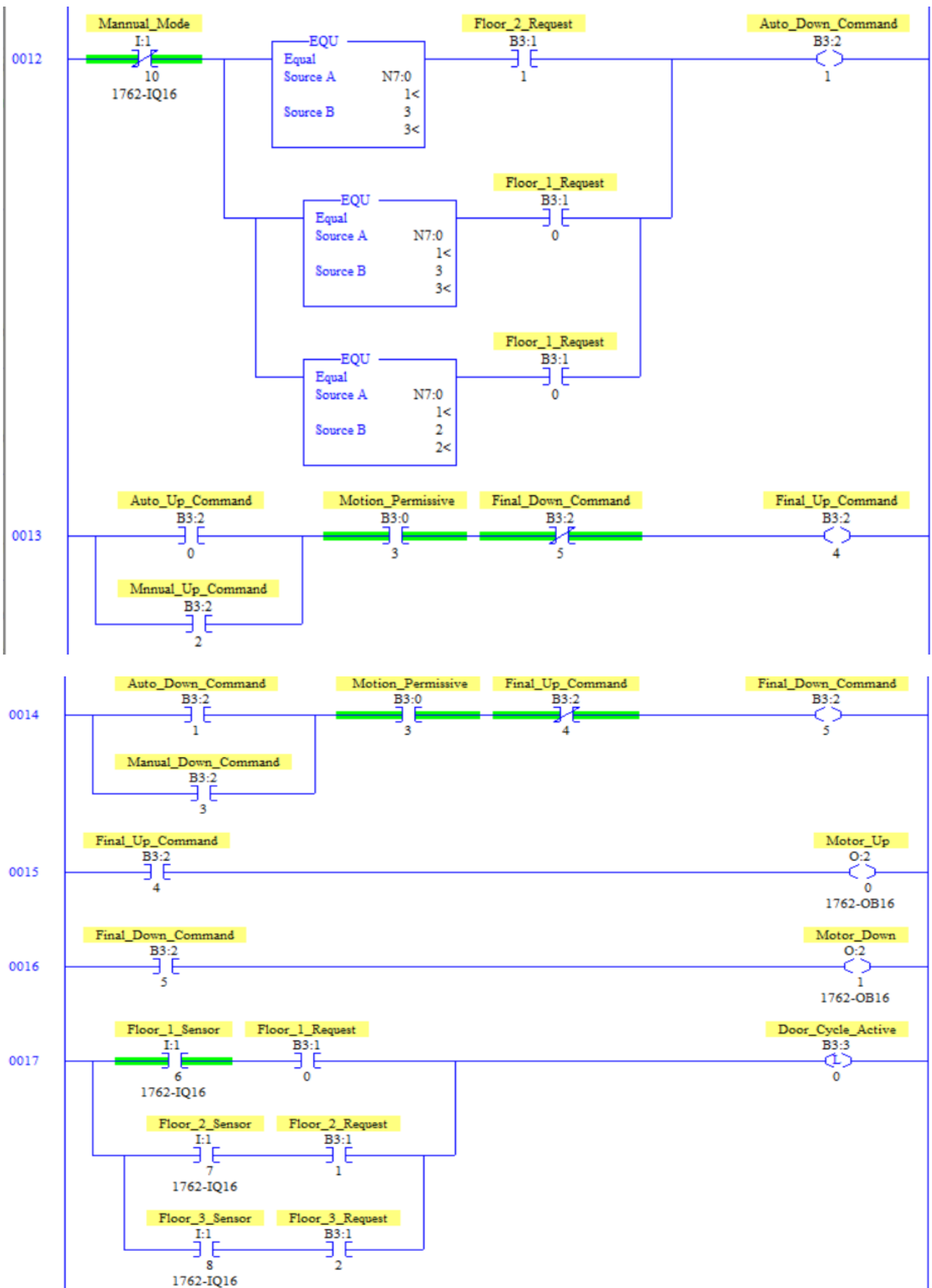


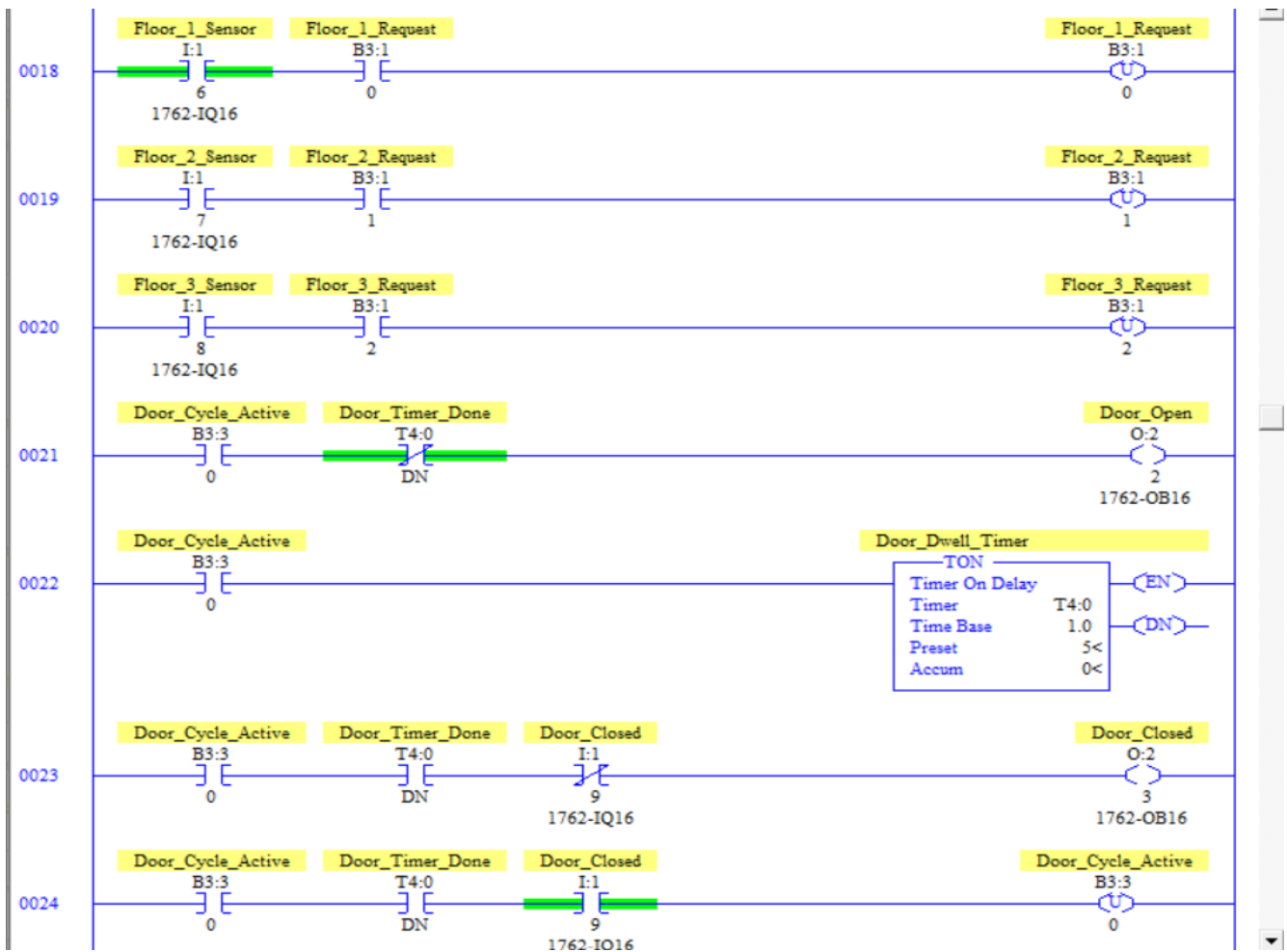
Engineering Purpose:

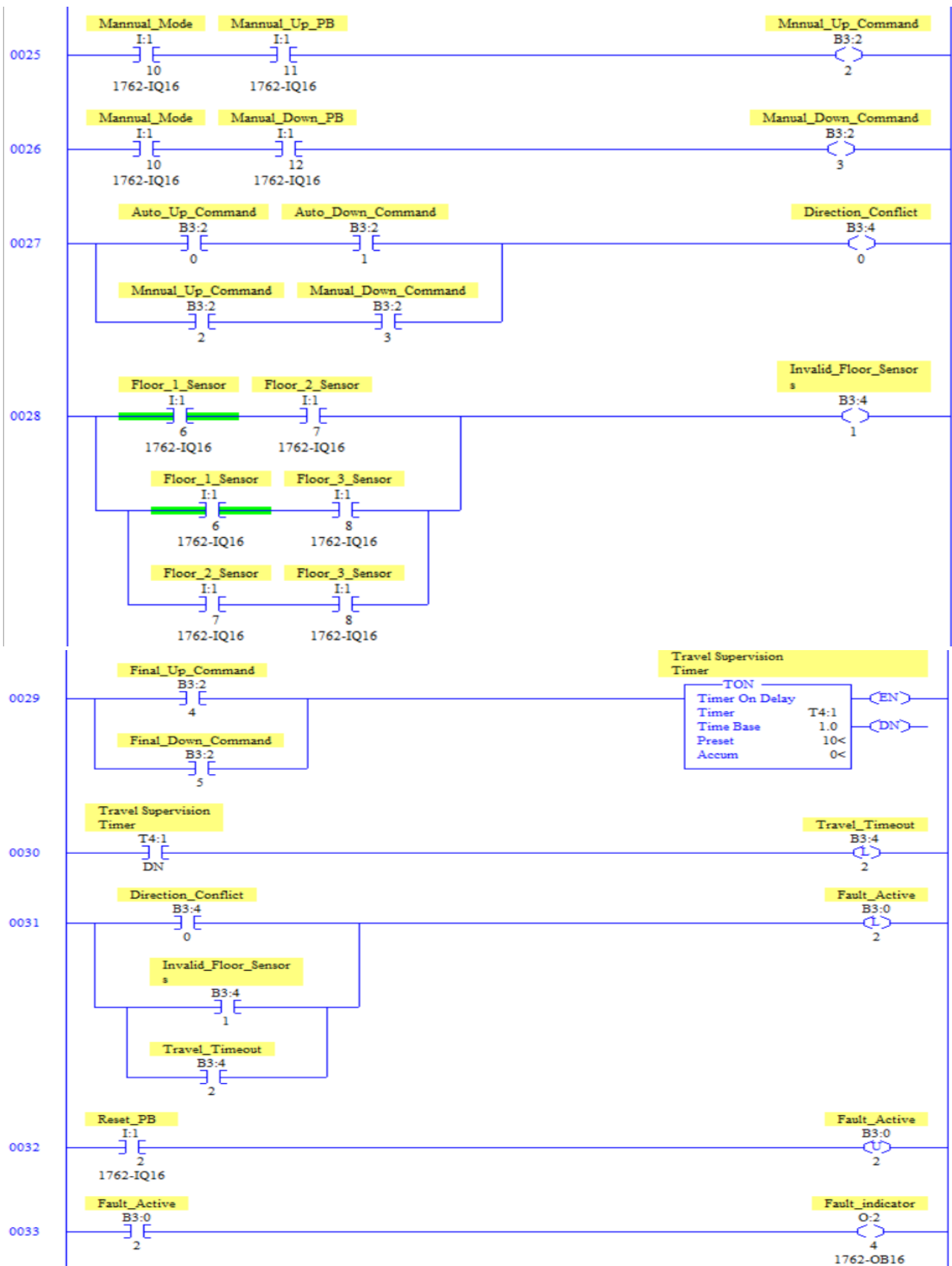
8.40 Actual RSLogix Implementation

Rack	Component	Input	Output
1	PS 307 5A		
2	CPU 314C-2 DP	I:1-I:16	Q:1-Q:16
3	DI16xDC24V/DO16xDC24V/0A	I:1-I:16	Q:1-Q:16
4	DI16xDC24V/DO16xDC24V/0A	I:1-I:16	Q:1-Q:16
5	DI16xDC24V/DO16xDC24V/0A	I:1-I:16	Q:1-Q:16









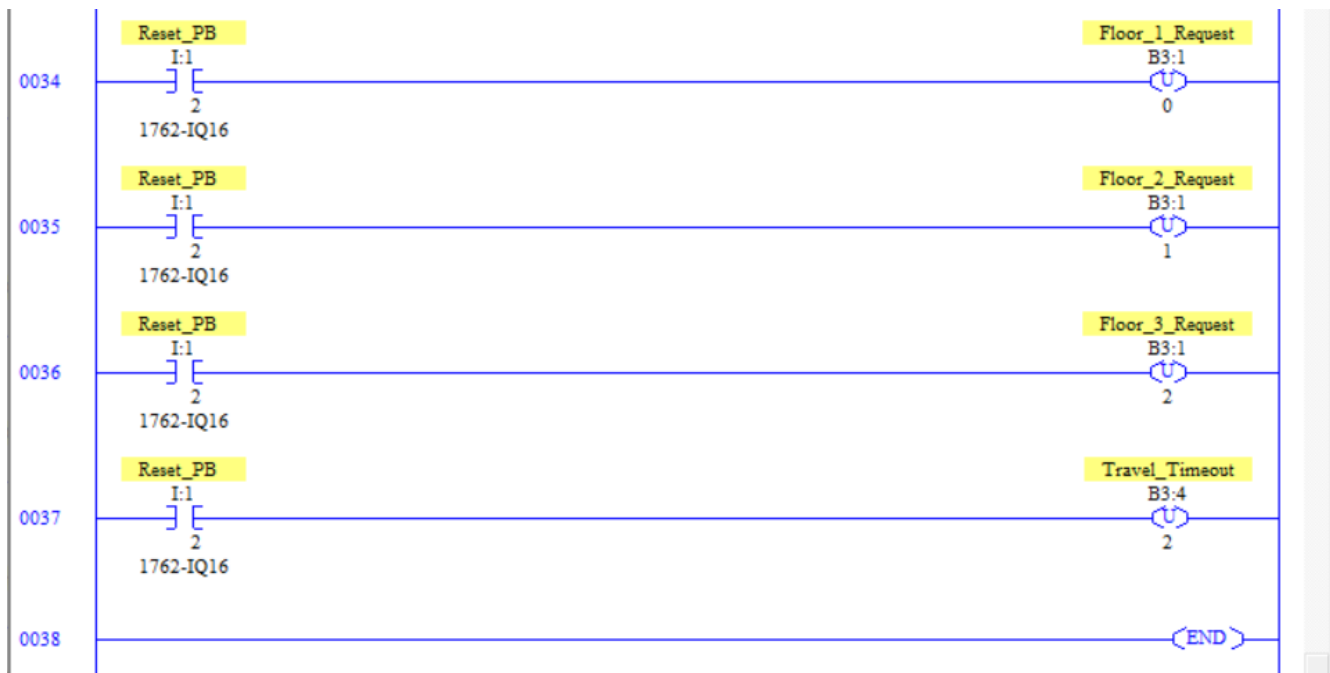


Figure P9V219 — Final RSLogix ladder screenshot.

9. Sequence of Operation

Step 1 — Start_PB latches Run_Request B3:0/0.

Step 2 — System_Ready B3:0/1 requires Run Request, Safety_OK, and no master fault.

Step 3 — Motion_Permissive B3:0/3 additionally requires Door_Closed and no active door cycle.

Step 4 — A floor-request pushbutton latches the corresponding internal destination request.

Step 5 — Floor sensors maintain the last confirmed position in N7:0.

Step 6 — Automatic logic compares N7:0 with the stored request and selects UP or DOWN while Manual Mode is off.

Step 7 — Automatic or Manual direction requests pass through Motion_Permissive and opposite-direction interlocks.

Step 8 — Final_Up_Command or Final_Down_Command energizes the corresponding physical motor output.

Step 9 — T4:1 supervises movement using TB 1.0 and PRE 60.

Step 10 — At the requested destination, the matching floor sensor/request pair latches Door_Cycle_Active before the request is cleared.

Step 11 — Door_Cycle_Active removes Motion_Permissive and energizes Door_Open while T4:0 is not done.

Step 12 — T4:0 uses TB 1.0 and PRE 20.

Step 13 — After T4:0/DN, Door_Close energizes until Door_Closed feedback returns.

Step 14 — Door_Cycle_Active then unlatches, T4:0 resets, and Motion_Permissive becomes available again.

Step 15 — Direction Conflict, Invalid Floor Sensors, or Travel Timeout can latch Fault_Active.

Step 16 — Fault Active removes Run Request and stops the system through the normal hierarchy.

Step 17 — Reset clears the master fault, stored floor requests, and Travel Timeout; a separate Start is then required.

10. Testing & Validation

Test	Verification	Result
Startup / Ready State	Run Request, System Ready, Motion Permissive	PASS
Floor 1 → Floor 2	Automatic upward travel and arrival	PASS
Floor 2 → Floor 3	Automatic upward travel and arrival	PASS
Floor 3 → Floor 1	Automatic downward travel and arrival	PASS
Door Sequence	Open, dwell, close, Door Closed confirmation	PASS
Manual UP	Manual command through common permissives	PASS
Manual DOWN	Manual command through common permissives	PASS
Door-Open Motion Interlock	Movement blocked with Door Closed false	PASS
Invalid Floor Sensors	Diagnostic and master fault generated	PASS
Travel Timeout	Latched timeout and controlled shutdown	PASS
Fault Reset	Fault/request recovery	PASS
Controlled Restart	Separate Start required after Reset	PASS
Final Integrated Acceptance	Healthy final controller state	PASS

Testing was performed in the PLC/emulator environment using manually simulated floor sensors, door feedback, mode selection, and operator commands. These results demonstrate ladder-program behavior, not passenger-elevator commissioning or safety certification.

11. Results

Project 9 Version 2 successfully combines position tracking, stored floor requests, automatic direction decisions, Manual Mode, door sequencing, travel supervision, diagnostic monitoring, fault shutdown, and controlled recovery in one coordinated PLC program.

The strongest result is the separation between request, decision, authorization, and physical movement. A floor request must become a stored request, produce a direction decision, pass Motion Permissive, and create a final command before a motor output can energize.

The door sequence also integrates arrival capture, motion inhibition, timed dwell, closing command, and Door Closed confirmation. The fault architecture uses individual diagnostics to feed a common master fault that removes the run request and forces controlled shutdown.

12. Challenges & Troubleshooting

Symptom	Diagnostic Path
System not ready	B3:0/0 → I:1/13 → B3:0/2 → B3:0/1
Motion not permitted	B3:0/1 → I:1/9 → B3:3/0 → B3:0/3
UP motor not running	Request → B3:2/0/B3:2/2 → B3:0/3 → B3:2/5 → B3:2/4 → O:2/0
DOWN motor not running	Request → B3:2/1/B3:2/3 → B3:0/3 → B3:2/4 → B3:2/5 → O:2/1
Wrong automatic direction	N7:0 → stored requests → Rungs 0011/0012
No door cycle at arrival	Destination sensor + stored request → B3:3/0
Door Open not active	B3:3/0 → T4:0/DN → O:2/2
Door Close not active	B3:3/0 → T4:0/DN → I:1/9 → O:2/3
Door cycle never completes	T4:0/DN → I:1/9 → B3:3/0
Invalid floor fault	Check I:1/6, I:1/7, I:1/8 combinations
Travel timeout	B3:2/4/B3:2/5 → T4:1 → B3:4/2
Fault returns after Reset	Check initiating diagnostic remains active
Unexpected request after recovery	Confirm B3:1/0–B3:1/2 cleared by Reset

Two particularly important commissioning lessons were the use of longer emulator timer values for observation and the need to latch Travel_Timeout because the master fault removes movement and resets the non-retentive travel timer.

13. Lessons Learned

This project strengthened my understanding of layered PLC control. Intermediate bits make troubleshooting clearer because I can identify whether a problem originates in request storage, position tracking, direction selection, motion permission, final commands, or physical outputs.

The use of N7:0 also demonstrated the value of retaining last-confirmed position when feedback is discrete rather than continuous. The project reinforced that interlocking and diagnostics are separate engineering functions: final-command mutual exclusion prevents contradictory motion, while Direction Conflict detects contradictory upstream requests.

The automatic door sequence reinforced the importance of real feedback. Timer completion alone does not restore motion; Door_Closed feedback must also be confirmed. Fault recovery similarly remains deliberate: Reset clears fault/request state, while Start is a separate action.

14. Industrial Relevance

Although this project is not a certified elevator controller, it demonstrates concepts relevant to hoists, lifts, transfer systems, vertical material handling, automated storage/retrieval equipment, and other industrial positioning systems. These include stored requests, position tracking, direction determination, common motion permissives, mutual interlocks, Automatic/Manual control, feedback-based sequencing, travel supervision, diagnostic faults, and controlled recovery.

A real passenger elevator would require certified safety systems, applicable codes and standards, redundant safety architecture, door-lock monitoring, overspeed protection, braking systems, emergency functions, and specialized hardware.

15. Version 1 → Version 2 Engineering Progression

Version 1 Foundation	Version 2 Implementation	Engineering Improvement
Basic floor movement	Stored floor requests	Destination retained after PB release
Direct floor logic	N7:0 Current_Floor	Last confirmed floor retained between sensors
Simple UP/DOWN decision	Auto UP/DOWN command layer	Separates decision from output
Limited Manual control	Separate Manual UP/DOWN commands	Controlled maintenance movement
Basic movement permissive	System Ready + Door Closed + Door Cycle interlock	Stronger motion authorization
Direct output logic	Final UP/DOWN command layer	Shared permissive and mutual interlocking
Limited direction protection	Direction Conflict diagnostic	Adds diagnostic visibility
Simple destination stop	Arrival capture before request clearing	Reliable scan-order transition
Basic door operation	Door Cycle Active + timer + feedback	Structured automatic door sequence
Limited position diagnostics	Invalid Floor Sensors	Detects contradictory feedback
No travel supervision	T4:1 + latched Travel Timeout	Detects excessive movement duration
Limited fault handling	Latched Fault Active	Controlled shutdown and recovery
Basic reset	Fault + request + timeout reset	Known post-fault state
Possible automatic recovery	Separate Start after Reset	Prevents automatic restart
Limited fault indication	Fault_Indicator O:2/4	Visible master-fault state

16. Project Summary

Project 9 Version 2 — Three-Floor Elevator Controller implements a structured PLC-based elevator simulation using an Allen-Bradley MicroLogix platform and RSLogix Micro Starter Lite. The final frozen program contains Rungs 0000–0037 plus END at Rung 0038.

The program includes Run Request, System Ready, Safety_OK, Motion Permissive, stored floor requests, N7:0 Current_Floor, automatic and manual direction commands, final UP/DOWN interlocks, Motor Up/Down outputs, arrival capture, automatic door sequencing, Direction Conflict, Invalid Floor Sensors, T4:1 travel supervision, latched Travel Timeout, master fault shutdown, fault indication, and controlled Reset.

The final screenshot-confirmed commissioning timer settings are T4:0 = TB 1.0 / PRE 20 and T4:1 = TB 1.0 / PRE 60. Testing documented successful startup, automatic travel in both directions, between-floor position retention, arrival, automatic door operation, Manual Mode, door-open motion inhibition, invalid-floor-sensor detection, travel timeout, fault shutdown, reset, controlled restart, and final integrated acceptance.

The project is documented as a PLC/emulator portfolio implementation, not as physical passenger-elevator commissioning or safety certification.